

CHAPTER 7

STACKS

IN THIS CHAPTER, YOU WILL:

- Learn about stacks
- Examine various stack operations
- Learn how to implement a stack as an array
- Learn how to implement a stack as a linked list
- Discover stack applications
- Learn how to use a stack to remove recursion

This chapter discusses a very useful data structure called a stack. It has numerous applications in computer science.

Stacks

Suppose that you have a program with several functions. To be specific, suppose that you have the functions A, B, C, and D in your program. Now suppose that function A calls function B, function B calls function C, and function C calls function D. When function D terminates, control goes back to function C; when function C terminates, control goes back to function B; and when function B terminates, control goes back to function A. During program execution, how do you think the computer keeps track of the function calls? What about recursive functions? How does the computer keep track of the recursive calls? In Chapter 6, we designed a recursive function to print a linked list backward. What if you want to write a nonrecursive algorithm to print a linked list backward?

This section discusses the data structure called the **stack**, which the computer uses to implement function calls. You can also use stacks to convert recursive algorithms into nonrecursive algorithms, especially recursive algorithms that are not tail recursive. Stacks have numerous other applications in computer science. After developing the tools necessary to implement a stack, we will examine some applications of stacks.

A stack is a list of homogenous elements in which the addition and deletion of elements occurs only at one end, called the **top** of the stack. For example, in a cafeteria, the second tray in a stack of trays can be removed only if the first tray has been removed. For another example, to get to your favorite computer science book, which is underneath your math and history books, you must first remove the math and history books. After removing these books, the computer science book becomes the top book—that is, the top element of the stack. Figure 7-1 shows some examples of stacks.

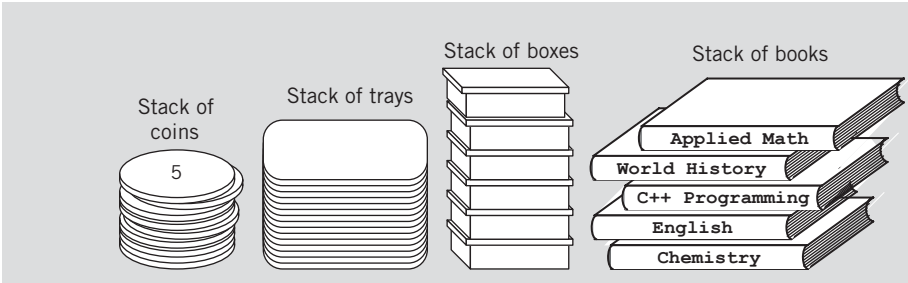


FIGURE 7-1 Various examples of stacks

The elements at the bottom of the stack have been in the stack the longest. The top element of the stack is the last element added to the stack. Because the elements are added

and removed from one end (that is, the top), it follows that the item that is added last will be removed first. For this reason, a stack is also called a **Last In First Out (LIFO)** data structure.

Stack: A data structure in which the elements are added and removed from one end only; a Last In First Out (LIFO) data structure.

Now that you know what a stack is, let us see what kinds of operations can be performed on a stack. Because new items can be added to the stack, we can perform the add operation, called **`push`**, to add an element onto the stack. Similarly, because the top item can be retrieved and/or removed from the stack, we can perform the operation **`top`** to retrieve the top element of the stack, and the operation **`pop`** to remove the top element from the stack.

The `push`, `top`, and `pop` operations work as follows: Suppose there are boxes lying on the floor that need to be stacked on a table. Initially, all of the boxes are on the floor and the stack is empty. (See Figure 7-2.)



FIGURE 7-2 Empty stack

First we push box `A` onto the stack. After the `push` operation, the stack is as shown in Figure 7-3(a).

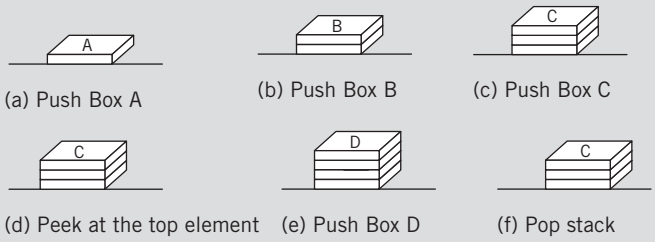


FIGURE 7-3 Stack operations

We then push box `B` onto the stack. After this `push` operation, the stack is as shown in Figure 7-3(b). Next, we push box `C` onto the stack. After this `push` operation, the stack is as shown in Figure 7-3(c). Next, we look at the top element of the stack. After this operation, the stack is unchanged and shown in Figure 7-3(d). We then push box `D` onto

the stack. After this **push** operation, the stack is as shown in Figure 7-3(e). Next, we pop the stack. After the **pop** operation, the stack is as shown in Figure 7-3(f).

An element can be removed from the stack only if there is something in the stack, and an element can be added to the stack only if there is room. The two operations that immediately follow from **push**, **top**, and **pop** are **isFullStack** (checks whether the stack is full) and **isEmptyStack** (checks whether the stack is empty). Because a stack keeps changing as we add and remove elements, the stack must be empty before we first start using it. Thus, we need another operation, called **initializeStack**, which initializes the stack to an empty state. Therefore, to successfully implement a stack, we need at least these six operations, which are described next. (We might also need other operations on a stack, depending on the specific implementation.)

- **initializeStack**—Initializes the stack to an empty state.
- **isEmptyStack**—Determines whether the stack is empty. If the stack is empty, it returns the value **true**; otherwise, it returns the value **false**.
- **isFullStack**—Determines whether the stack is full. If the stack is full, it returns the value **true**; otherwise, it returns the value **false**.
- **push**—Adds a new element to the top of the stack. The input to this operation consists of the stack and the new element. Prior to this operation, the stack must exist and must not be full.
- **top**—Returns the top element of the stack. Prior to this operation, the stack must exist and must not be empty.
- **pop**—Removes the top element of the stack. Prior to this operation, the stack must exist and must not be empty.

The following abstract **class stackADT** defines these operations as an ADT:

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the basic operations on a stack.
//*************************************************************

template <class Type>
class stackADT
{
public:
    virtual void initializeStack() = 0;
       //Method to initialize the stack to an empty state.
       //Postcondition: Stack is empty.

    virtual bool isEmptyStack() const = 0;
      //Function to determine whether the stack is empty.
      //Postcondition: Returns true if the stack is empty,
      //    otherwise returns false.
```

```
    virtual bool isFullStack() const = 0;
      //Function to determine whether the stack is full.
      //Postcondition: Returns true if the stack is full,
      //    otherwise returns false.

    virtual void push(const Type& newItem) = 0;
      //Function to add newItem to the stack.
      //Precondition: The stack exists and is not full.
      //Postcondition: The stack is changed and newItem is added
      //    to the top of the stack.

    virtual Type top() const = 0;
      //Function to return the top element of the stack.
      //Precondition: The stack exists and is not empty.
      //Postcondition: If the stack is empty, the program
      //    terminates; otherwise, the top element of the stack
      //    is returned.

    virtual void pop() = 0;
      //Function to remove the top element of the stack.
      //Precondition: The stack exists and is not empty.
      //Postcondition: The stack is changed and the top element
      //    is removed from the stack.
};
```

Figure 7-4 shows the UML class diagram of the **class stackADT**.

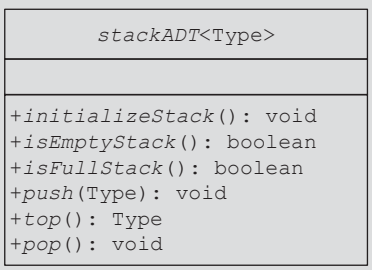


FIGURE 7-4 UML class diagram of the `class stackADT`

We now consider the implementation of our abstract stack data structure. Functions such as **push** and **pop** that are required to implement a stack are not available to C++ programmers. We must write the functions to implement the stack operations.

Because all the elements of a stack are of the same type, a stack can be implemented as either an array or a linked structure. Both implementations are useful and are discussed in this chapter.

Implementation of Stacks as Arrays

Because all the elements of a stack are of the same type, you can use an array to implement a stack. The first element of the stack can be put in the first array slot, the second element of the stack in the second array slot, and so on. The top of the stack is the index of the last element added to the stack.

In this implementation of a stack, stack elements are stored in an array, and an array is a random access data structure; that is, you can directly access any element of the array. However, by definition, a stack is a data structure in which the elements are accessed (popped or pushed) at only one end—that is, a Last In First Out data structure. Thus, a stack element is accessed only through the top, not through the bottom or middle. This feature of a stack is extremely important and must be recognized in the beginning.

To keep track of the top position of the array, we can simply declare another variable, called **stackTop**.

The following **class**, **stackType**, implements the functions of the abstract **class stackADT**. By using a pointer, we can dynamically allocate arrays, so we will leave it for the user to specify the size of the array (that is, the stack size). We assume that the default stack size is 100. Because the **class stackType** has a pointer member variable (the pointer to the array to store the stack elements), we must overload the assignment operator and include the copy constructor and destructor. Moreover, we give a generic definition of the stack. Depending on the specific application, we can pass the stack element type when we declare a stack object.

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the basic operation on a stack as an
// array.
//*************************************************************

template <class Type>
class stackType: public stackADT<Type>
{
public:
    const stackType<Type>& operator=(const stackType<Type>&);
      //Overload the assignment operator.

    void initializeStack();
      //Function to initialize the stack to an empty state.
      //Postcondition: stackTop = 0;

    bool isEmptyStack() const;
      //Function to determine whether the stack is empty.
      //Postcondition: Returns true if the stack is empty,
      //    otherwise returns false.
```

```
    bool isFullStack() const;
      //Function to determine whether the stack is full.
      //Postcondition: Returns true if the stack is full,
      //    otherwise returns false.

    void push(const Type& newItem);
      //Function to add newItem to the stack.
      //Precondition: The stack exists and is not full.
      //Postcondition: The stack is changed and newItem is
      //    added to the top of the stack.

    Type top() const;
      //Function to return the top element of the stack.
      //Precondition: The stack exists and is not empty.
      //Postcondition: If the stack is empty, the program
      //    terminates; otherwise, the top element of the stack
      //    is returned.

    void pop();
      //Function to remove the top element of the stack.
      //Precondition: The stack exists and is not empty.
      //Postcondition: The stack is changed and the top element is
      //    removed from the stack.

    stackType(int stackSize = 100);
      //Constructor
      //Create an array of the size stackSize to hold
      //the stack elements. The default stack size is 100.
      //Postcondition: The variable list contains the base address
      //   of the array, stackTop = 0, and maxStackSize = stackSize

    stackType(const stackType<Type>& otherStack);
      //Copy constructor

    ~stackType();
      //Destructor
      //Remove all the elements from the stack.
      //Postcondition: The array (list) holding the stack
      //    elements is deleted.

private:
    int maxStackSize; //variable to store the maximum stack size
    int stackTop;     //variable to point to the top of the stack
    Type *list; //pointer to the array that holds the stack elements

    void copyStack(const stackType<Type>& otherStack);
      //Function to make a copy of otherStack.
      //Postcondition: A copy of otherStack is created and assigned
      //    to this stack.
};
```

Figure 7-5 shows the UML class diagram of the **class stackType**.

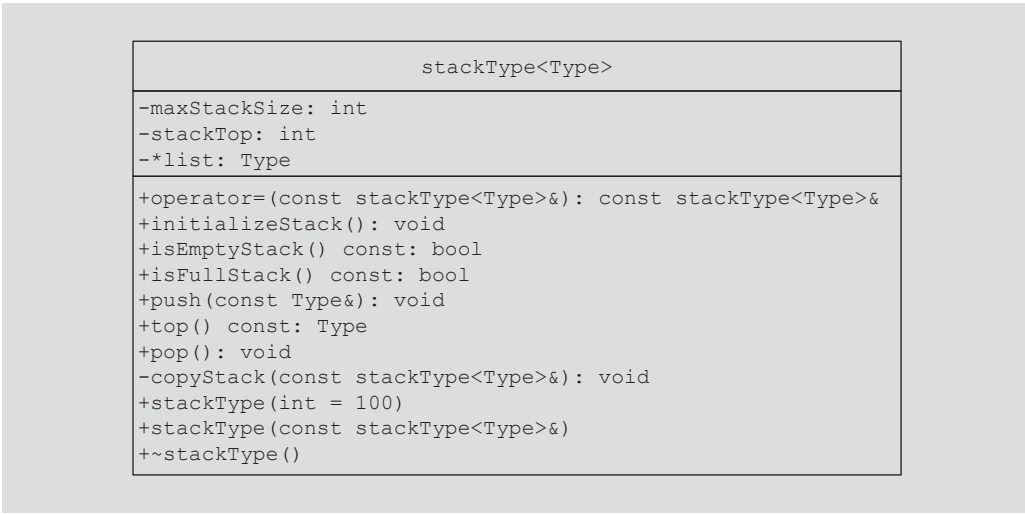


FIGURE 7-5 UML class diagram of the `class stackType`

NOTE

If **stackTop** is 0, the stack is empty. If **stackTop** is nonzero, the stack is nonempty and the top element of the stack is given by **stackTop – 1** because the first stack element is at position 0.

NOTE

The function **copyStack** is included as a **private** member. It contains the code that is common to the functions to overload the assignment operator and the copy constructor. We use this function only to implement the copy constructor and overload the assignment operator. To copy a stack into another stack, the program can use the assignment operator.

Figure 7-6 shows this data structure, wherein **stack** is an object of type **stackType**. Note that **stackTop** can range from 0 to **maxStackSize**. If **stackTop** is nonzero, then **stackTop – 1** is the index of the top element of the stack. Suppose that **maxStackSize = 100**.

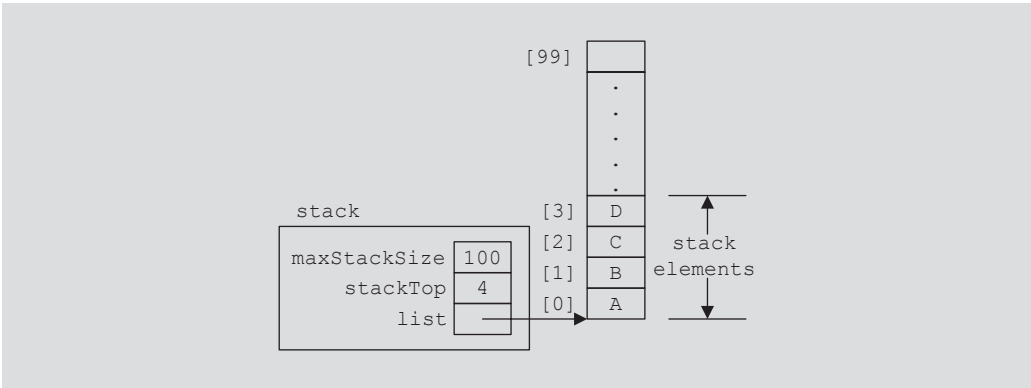


FIGURE 7-6 Example of a stack

Note that the pointer `list` contains the base address of the array (holding the stack elements)—that is, the address of the first array component. Next we discuss how to implement the member functions of the `class stackType`.

Initialize Stack

Let us consider the `initializeStack` operation. Because the value of `stackTop` indicates whether the stack is empty, we can simply set `stackTop` to `0` to initialize the stack. (See Figure 7-7.)

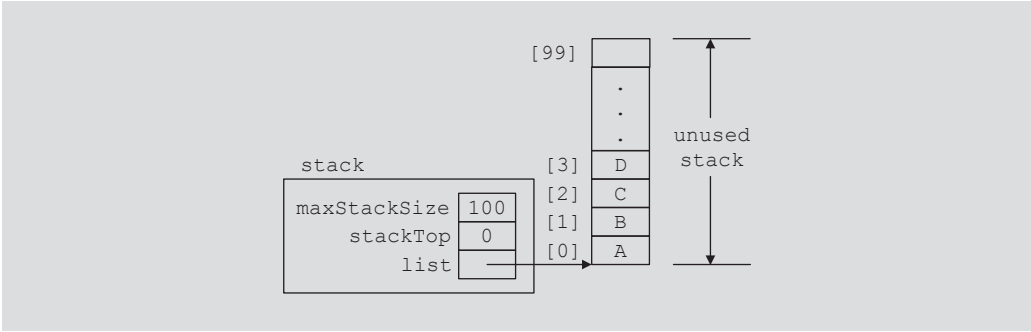


FIGURE 7-7 Empty stack

The definition of the function `initializeStack` is as follows:

```
template <class Type>
void stackType<Type>::initializeStack()
{
    stackTop = 0;
}//end initializeStack
```

Empty Stack

We have seen that the value of `stackTop` indicates whether the stack is empty. If `stackTop` is `0`, the stack is empty; otherwise, the stack is not empty. The definition of the function `isEmptyStack` is as follows:

```
template <class Type>
bool stackType<Type>::isEmptyStack() const
{
    return(stackTop == 0);
}//end isEmptyStack
```

Full Stack

Next, we consider the operation `isFullStack`. It follows that the stack is full if `stackTop` is equal to `maxStackSize`. The definition of the function `isFullStack` is as follows:

```
template <class Type>
bool stackType<Type>::isFullStack() const
{
    return(stackTop == maxStackSize);
} //end isFullStack
```

Push

Adding, or pushing, an element onto the stack is a two-step process. Recall that the value of `stackTop` indicates the number of elements in the stack, and `stackTop - 1` gives the position of the top element of the stack. Therefore, the `push` operation is as follows:

1. Store the `newItem` in the array component indicated by `stackTop`.
2. Increment `stackTop`.

Figure 7-8(a) shows the stack before pushing `'y'` into the stack. Figure 7-8(b) shows the stack after pushing `'y'` into the stack.

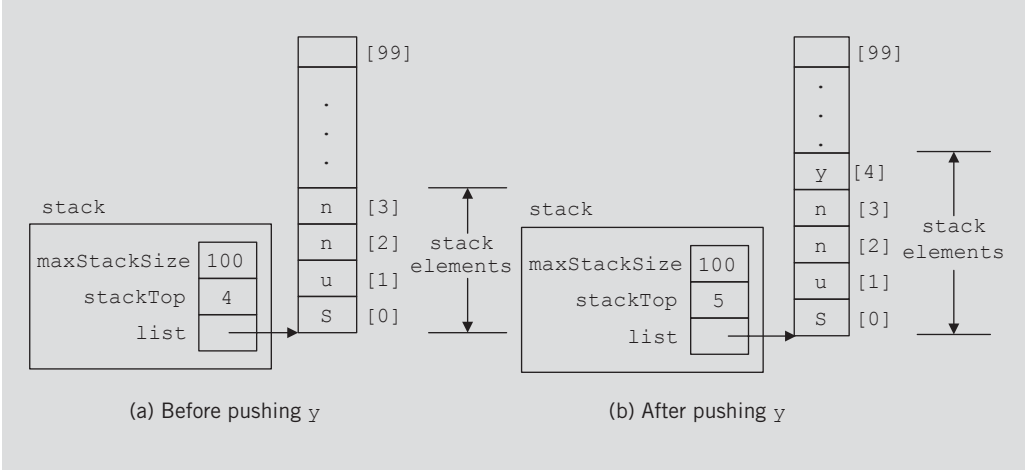


FIGURE 7-8 Stack before and after the `push` operation

The definition of the function `push` is as follows:

```
template <class Type>
void stackType<Type>::push(const Type& newItem)
{
    if (!isFullStack())
    {
        list[stackTop] = newItem;  //add newItem at the top
        stackTop++; //increment stackTop
    }
    else
        cout << "Cannot add to a full stack." << endl;
}//end push
```

If we try to add a new item to a full stack, the resulting condition is called an **overflow**. Error checking for an overflow can be handled in different ways. One way is as shown previously. Or, we can check for an overflow before calling the function `push`, as shown next (assuming `stack` is an object of type `stackType`).

```
if (!stack.isFullStack())
    stack.push(newItem);
```


Return the Top Element

The operation `top` returns the top element of the stack. Its definition is as follows:

```
template <class Type>
Type stackType<Type>::top() const
{
    assert(stackTop != 0);  //if stack is empty, terminate the
                            //program
    return list[stackTop - 1]; //return the element of the stack
                               //indicated by stackTop - 1
}//end top
```

Pop

To remove, or pop, an element from the stack, we simply decrement `stackTop` by `1`.

Figure 7-9(a) shows the stack before popping `'D'` from the stack. Figure 7-9(b) shows the stack after popping `'D'` from the stack.

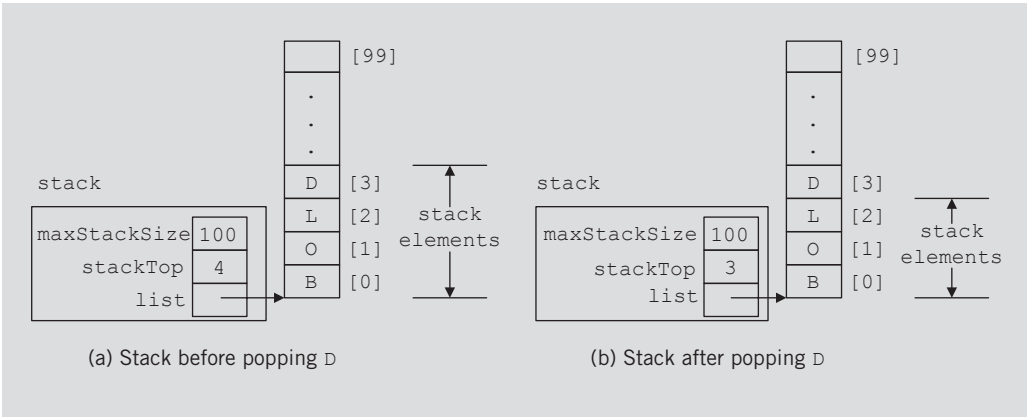


FIGURE 7-9 Stack before and after the pop operation

The definition of the function **pop** is as follows:

```
template <class Type>
void stackType<Type>::pop()
{
    if (!isEmptyStack())
        stackTop--;     //decrement stackTop
    else
        cout << "Cannot remove from an empty stack." << endl;
}//end pop
```

If we try to remove an item from an empty stack, the resulting condition is called an **underflow**. Error checking for an underflow can be handled in different ways. One way is as shown previously. Or, we can check for an underflow before calling the function **pop**, as shown next (assuming **stack** is an object of type **stackType**).

```
if (!stack.isEmptyStack())
    stack.pop();
```

Copy Stack

The function **copyStack** makes a copy of a stack. The stack to be copied is passed as a parameter to the function **copyStack**. We will, in fact, use this function to implement the copy constructor and overload the assignment operator. The definition of this function is as follows:

```
template <class Type>
void stackType<Type>::copyStack(const stackType<Type>& otherStack)
{
    delete [] list;
    maxStackSize = otherStack.maxStackSize;
    stackTop = otherStack.stackTop;
```

```
    list = new Type[maxStackSize];

        //copy otherStack into this stack
    for (int j = 0; j < stackTop; j++)
        list[j] = otherStack.list[j];
} //end copyStack
```

Constructor and Destructor

The functions to implement the constructor and the destructor are straightforward. The constructor with parameters sets the stack size to the size specified by the user, sets `stackTop` to `0`, and creates an appropriate array in which to store the stack elements. If the user does not specify the size of the array in which to store the stack elements, the constructor uses the default value, which is `100`, to create an array of size `100`. The destructor simply deallocates the memory occupied by the array (that is, the stack) and sets `stackTop` to `0`. The definitions of the constructor and destructor are as follows:

```
template <class Type>
stackType<Type>::stackType(int stackSize)
{
    if (stackSize <= 0)
    {
        cout << "Size of the array to hold the stack must "
             << "be positive." << endl;
        cout << "Creating an array of size 100." << endl;

        maxStackSize = 100;
    }
    else
        maxStackSize = stackSize;   //set the stack size to
                                    //the value specified by
                                    //the parameter stackSize

    stackTop = 0;                   //set stackTop to 0
    list = new Type[maxStackSize];  //create the array to
                                    //hold the stack elements
}//end constructor

template <class Type>
stackType<Type>::~stackType() //destructor
{
    delete [] list; //deallocate the memory occupied
                    //by the array
}//end destructor
```

Copy Constructor

The copy constructor is called when a stack object is passed as a (value) parameter to a function. It copies the values of the member variables of the actual parameter into the corresponding member variables of the formal parameter. Its definition is as follows:

```
template <class Type>
stackType<Type>::stackType(const stackType<Type>& otherStack)
{
    list = NULL;

    copyStack(otherStack);
}//end copy constructor
```

Overloading the Assignment Operator (=)

Recall that for classes with pointer member variables, the assignment operator must be explicitly overloaded. The definition of the function to overload the assignment operator for the `class stackType` is as follows:

```
template <class Type>
const stackType<Type>& stackType<Type>::operator=
                          (const stackType<Type>& otherStack)
{
    if (this != &otherStack) //avoid self-copy
        copyStack(otherStack);

    return *this;
} //end operator=
```

Stack Header File

Now that you know how to implement the stack operations, you can put the definitions of the class and the functions to implement the stack operations together to create the stack header file. For the sake of completeness, we next describe the header file. Suppose that the name of the header file containing the definition of the `class stackType` is `myStack.h`. We will refer to this header file in any program that uses a stack.

```
//Header file: myStack.h

#ifndef H_StackType
#define H_StackType

#include <iostream>
#include <cassert>

#include "stackADT.h"

using namespace std;

//Place the definition of the class template stackType, as given
//previously in this chapter, here.

//Place the definitions of the member functions as discussed here.
#endif
```

The analysis of the stack operations is similar to the operations of the `class arrayListType` (Chapter 3). We, therefore, provide only a summary in Table 7-1.

TABLE 7-1 Time complexity of the operations of the `class stackType` on a stack with *n* elements

Function	Time complexity
`isEmptyStack`	$O(1)$
`isFullStack`	$O(1)$
`initializeStack`	$O(1)$
constructor	$O(1)$
`top`	$O(1)$
`push`	$O(1)$
`pop`	$O(1)$
`copyStack`	$O(n)$
destructor	$O(1)$
copy constructor	$O(n)$
Overloading the assignment operator	$O(n)$

EXAMPLE 7-1

Before we give a programming example, let us first write a simple program that uses the **`class stackType`** and tests some of the stack operations. Among others, we will test the assignment operator and the copy constructor. The program and its output are as follows:

```
//***********************************************************
// Author: D.S. Malik
//
// This program tests various operations of a stack.
//***********************************************************

#include <iostream>
#include "myStack.h"

using namespace std;
```

```
void testCopyConstructor(stackType<int> otherStack);

int main()
{
    stackType<int> stack(50);
    stackType<int> copyStack(50);
    stackType<int> dummyStack(100);

    stack.initializeStack();
    stack.push(23);
    stack.push(45);
    stack.push(38);
    copyStack = stack;  //copy stack into copyStack

    cout << "The elements of copyStack: ";

    while (!copyStack.isEmptyStack())  //print copyStack
    {
        cout << copyStack.top() << " ";
        copyStack.pop();
    }
    cout << endl;

    copyStack = stack;
    testCopyConstructor(stack);  //test the copy constructor

    if (!stack.isEmptyStack())
        cout << "The original stack is not empty." << endl
             << "The top element of the original stack: "
             << copyStack.top() << endl;

    dummyStack = stack;  //copy stack into dummyStack

    cout << "The elements of dummyStack: ";

    while (!dummyStack.isEmptyStack())  //print dummyStack
    {
        cout << dummyStack.top() << " ";
        dummyStack.pop();
    }

    cout << endl;

    return 0;
}

void testCopyConstructor(stackType<int> otherStack)
{
    if (!otherStack.isEmptyStack())
        cout << "otherStack is not empty." << endl
             << "The top element of otherStack: "
             << otherStack.top() << endl;
}
```

Sample Run:

```
The elements of copyStack: 38 45 23
otherStack is not empty.
The top element of otherStack: 38
The original stack is not empty.
The top element of the original stack: 38
The elements of dummyStack: 38 45 23
```

It is recommended that you do a walk-through of this program.

PROGRAMMING EXAMPLE: Highest GPA

In this example, we write a C++ program that reads a data file consisting of each student's GPA followed by the student's name. The program then prints the highest GPA and the names of all the students who received that GPA. The program scans the input file only once. Moreover, we assume that there are a maximum of 100 students in the class.

Input The program reads an input file consisting of each student's GPA, followed by the student's name. Sample data is as follows:

```
3.5 Bill
3.6 John
2.7 Lisa
3.9 Kathy
3.4 Jason
3.9 David
3.4 Jack
```

Output The program outputs the highest GPA and all the names associated with the highest GPA. For example, for the preceding data, the highest GPA is `3.9` and the students with that GPA are `Kathy` and `David`.

PROGRAM ANALYSIS AND ALGORITHM DESIGN

We read the first GPA and the name of the student. Because this data is the first item read, it is the highest GPA so far. Next, we read the second GPA and the name of the student. We then compare this (second) GPA with the highest GPA so far. Three cases arise:

1. The new GPA is greater than the highest GPA so far. In this case, we do the following:
   a. Update the value of the highest GPA so far.
   b. Initialize the stack—that is, remove the names of the students from the stack.
   c. Save the name of the student having the highest GPA so far in the stack.

2. The new GPA is equal to the highest GPA so far. In this case, we add the name of the new student to the stack.
3. The new GPA is smaller than the highest GPA so far. In this case, we discard the name of the student having this grade.

We then read the next GPA and the name of the student, and repeat Steps 1 through 3. We continue this process until we reach the end of file.

From this discussion, it is clear that we need the following variables:

```
double GPA;          //variable to hold the current GPA
double highestGPA;   //variable to hold the highest GPA
string name;         //variable to hold the name of the student
stackType<string> stack(100); //object to implement the stack
```

The previous discussion translates into the following algorithm:

1. Declare the variables and initialize stack.
2. Open the input file.
3. If the input file does not exist, exit the program.
4. Set the output of the floating-point numbers to a fixed decimal format with a decimal point and trailing zeroes. Also, set the precision to two decimal places.
5. Read the GPA and the student name.
6. `highestGPA = GPA;`
7. `while` (not end of file)
   `{`
   7.1. `if (GPA > highestGPA)`
   `{`
   7.1.1. `initializeStack(stack);`
   7.1.2. `push(stack, student name);`
   7.1.3. `highestGPA = GPA;`
   `}`
   7.2. `else`
   `if (GPA` is equal to `highestGPA)`
   `push(stack, student name);`
   7.3. Read `GPA` and `student name;`
   `}`
8. Output the highest GPA.
9. Output the names of the students having the highest GPA.

PROGRAM LISTING

```
//***************************************************************
// Author: D.S. Malik
//
// This program reads a data file consisting of students' GPAs
// followed by their names. The program then prints the highest
// GPA and the names of the students with the highest GPA.
//***************************************************************

#include <iostream>
#include <iomanip>
#include <fstream>
#include <string>

#include "myStack.h"

using namespace std;

int main()
{
        //Step 1
    double GPA;
    double highestGPA;
    string name;

    stackType<string> stack(100);

    ifstream infile;

    infile.open("HighestGPAData.txt");          //Step 2

    if (!infile)                                //Step 3
    {
        cout << "The input file does not "
             << "exist. Program terminates!" << endl;
        return 1;
    }

    cout << fixed << showpoint;                 //Step 4
    cout << setprecision(2);                    //Step 4

    infile >> GPA >> name;                      //Step 5

    highestGPA = GPA;                           //Step 6

    while (infile)                              //Step 7
    {
        if (GPA > highestGPA)                   //Step 7.1
        {
            stack.initializeStack();            //Step 7.1.1
```

```
            if (!stack.isFullStack())           //Step 7.1.2
                stack.push(name);

            highestGPA = GPA;                   //Step 7.1.3
        }
        else if (GPA == highestGPA)             //Step 7.2
            if (!stack.isFullStack())
                stack.push(name);
            else
            {
                cout << "Stack overflows. "
                     << "Program terminates!" << endl;
                return 1;  //exit program
            }

        infile >> GPA >> name;                  //Step 7.3
    }

    cout << "Highest GPA = " << highestGPA << endl;//Step 8
    cout << "The students holding the "
         << "highest GPA are:" << endl;

    while (!stack.isEmptyStack())                   //Step 9
    {
        cout << stack.top() << endl;
        stack.pop();
    }

    cout << endl;

    return 0;
}
```

Sample Run:

Input File (HighestGPAData.txt)

```
3.4 Randy
3.2 Kathy
2.5 Colt
3.4 Tom
3.8 Ron
3.8 Mickey
3.6 Peter
3.5 Donald
3.8 Cindy
3.7 Dome
3.9 Andy
3.8 Fox
3.9 Minnie
2.7 Gilda
3.9 Vinay
3.4 Danny
```

Output

```
Highest GPA = 3.90
The students holding the highest GPA are:
Vinay
Minnie
Andy
```

Note that the names of the students with the highest GPA are output in the reverse order, relative to the order they appear in the input because the top element of the stack is the last element added to the stack.

Linked Implementation of Stacks

Because an array size is fixed, in the array (linear) representation of a stack, only a fixed number of elements can be pushed onto the stack. If in a program the number of elements to be pushed exceeds the size of the array, the program might terminate in an error. We must overcome this problem.

We have seen that by using pointer variables we can dynamically allocate and deallocate memory, and by using linked lists we can dynamically organize data (such as an ordered list). Next, we will use these concepts to implement a stack dynamically.

Recall that in the linear representation of a stack, the value of `stackTop` indicates the number of elements in the stack, and the value of `stackTop - 1` points to the top item in the stack. With the help of `stackTop`, we can do several things: Find the top element, check whether the stack is empty, and so on.

Similar to the linear representation, in a linked representation `stackTop` is used to locate the top element in the stack. However, there is a slight difference. In the former case, `stackTop` gives the index of the array. In the latter case, `stackTop` gives the address (memory location) of the top element of the stack.

The following class implements the functions of the abstract `class stackADT`:

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the basic operation on a stack as a
// linked list.
//*************************************************************

    //Definition of the node
template <class Type>
struct nodeType
{
    Type info;
    nodeType<Type> *link;
};
```

```
template <class Type>
class linkedStackType: public stackADT<Type>
{
public:
    const linkedStackType<Type>& operator=
                              (const linkedStackType<Type>&);
      //Overload the assignment operator.

    bool isEmptyStack() const;
      //Function to determine whether the stack is empty.
      //Postcondition: Returns true if the stack is empty;
      //    otherwise returns false.

    bool isFullStack() const;
      //Function to determine whether the stack is full.
      //Postcondition: Returns false.

    void initializeStack();
      //Function to initialize the stack to an empty state.
      //Postcondition: The stack elements are removed;
      //    stackTop = NULL;

    void push(const Type& newItem);
      //Function to add newItem to the stack.
      //Precondition: The stack exists and is not full.
      //Postcondition: The stack is changed and newItem is
      //    added to the top of the stack.

    Type top() const;
      //Function to return the top element of the stack.
      //Precondition: The stack exists and is not empty.
      //Postcondition: If the stack is empty, the program
      //    terminates; otherwise, the top element of
      //    the stack is returned.

    void pop();
      //Function to remove the top element of the stack.
      //Precondition: The stack exists and is not empty.
      //Postcondition: The stack is changed and the top
      //    element is removed from the stack.

    linkedStackType();
      //Default constructor
      //Postcondition: stackTop = NULL;

    linkedStackType(const linkedStackType<Type>& otherStack);
      //Copy constructor

    ~linkedStackType();
      //Destructor
      //Postcondition: All the elements of the stack are removed.
```

```
private:
    nodeType<Type> *stackTop; //pointer to the stack

    void copyStack(const linkedStackType<Type>& otherStack);
      //Function to make a copy of otherStack.
      //Postcondition: A copy of otherStack is created and
      //    assigned to this stack.
};
```

NOTE

In this linked implementation of stacks, the memory to store the stack elements is allocated dynamically. Logically, the stack is never full. The stack is full only if we run out of memory space. Therefore, in reality, the function `isFullStack` does not apply to linked implementations of stacks. However, the `class linkedStackType` must provide the definition of the function `isFullStack` because it is defined in the parent abstract `class stackADT`.

We leave the UML class diagram of the `class linkedStackType` as an exercise for you. (See Exercise 12 at the end of this chapter.)

EXAMPLE 7-2

Suppose that `stack` is an object of type `linkedStackType`. Figure 7-10(a) shows an empty stack and Figure 7-10(b) shows a nonempty stack.

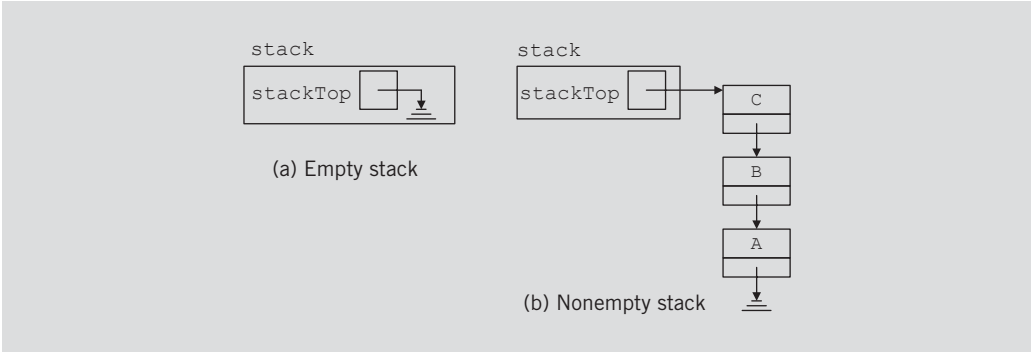


FIGURE 7-10 Empty and nonempty linked stacks

In Figure 7-10(b), the top element of the stack is `C`; that is, the last element pushed onto the stack is `C`.

Next, we discuss the definitions of the functions to implement the operations of a linked stack.

Default Constructor

The first operation that we consider is the default constructor. The default constructor initializes the stack to an empty state when a stack object is declared. Thus, this function sets `stackTop` to `NULL`. The definition of this function is as follows:

```
template  <class Type>
linkedStackType<Type>::linkedStackType()
{
    stackTop = NULL;
}
```

Empty Stack and Full Stack

The operations `isEmptyStack` and `isFullStack` are quite straightforward. The stack is empty if `stackTop` is `NULL`. Also, because the memory for a stack element is allocated and deallocated dynamically, the stack is never full. (The stack is full only if we run out of memory.) Thus, the function `isFullStack` always returns the value `false`. The definitions of the functions to implement these operations are as follows:

```
template <class Type>
bool linkedStackType<Type>::isEmptyStack() const
{
    return(stackTop == NULL);
} //end isEmptyStack

template <class Type>
bool linkedStackType<Type>::isFullStack() const
{
    return false;
} //end isFullStack
```

Recall that in the linked implementation of stacks, the function `isFullStack` does not apply because logically the stack is never full. However, you must provide its definition because it is included as an abstract function in the parent `class stackADT`.

Initialize Stack

The operation `initializeStack` reinitializes the stack to an empty state. Because the stack might contain some elements and we are using a linked implementation of a stack, we must deallocate the memory occupied by the stack elements and set `stackTop` to `NULL`. The definition of this function is as follows:

```
template <class Type>
void linkedStackType<Type>:: initializeStack()
{
    nodeType<Type> *temp; //pointer to delete the node

    while (stackTop != NULL)  //while there are elements in
                              //the stack
    {
        temp = stackTop;    //set temp to point to the
                            //current node
        stackTop = stackTop->link;  //advance stackTop to the
                                    //next node
        delete temp;    //deallocate memory occupied by temp
    }
} //end initializeStack
```

Next, we consider the `push`, `top`, and `pop` operations. From Figure 7-10(b), it is clear that the `newElement` will be added (in the case of `push`) at the beginning of the linked list pointed to by `stackTop`. In the case of `pop`, the node pointed to by `stackTop` will be removed. In both cases, the value of the pointer `stackTop` is updated. The operation `top` returns the `info` of the node to which `stackTop` is pointing.

Push

Consider the stack shown in Figure 7-11.

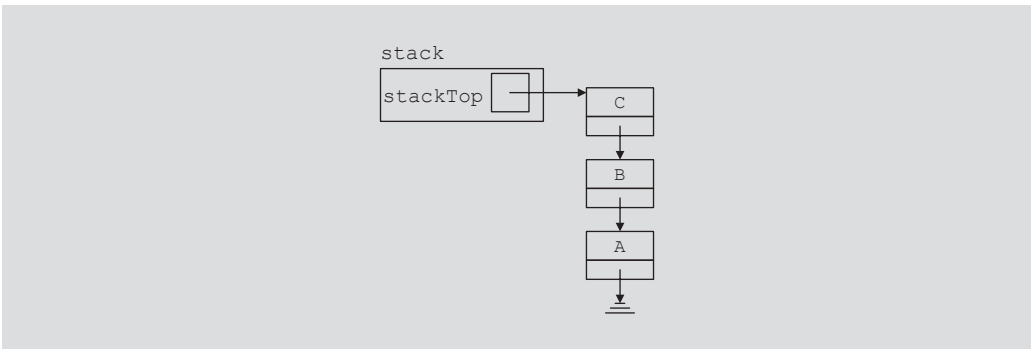


FIGURE 7-11 Stack before the `push` operation

Figure 7-12 shows the steps of the `push` operation. (Assume that the new element to be pushed is `'D'`.)

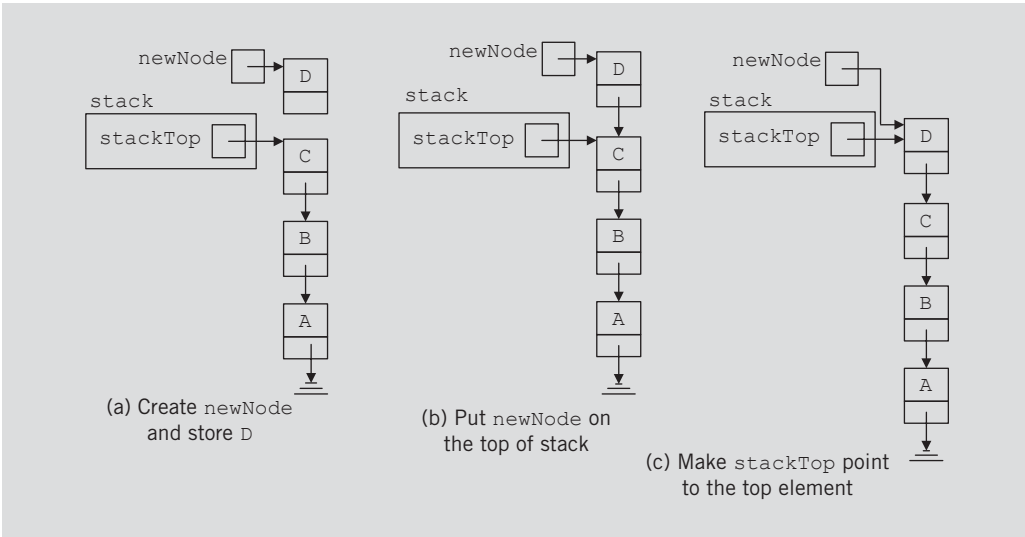


FIGURE 7-12 Push operation

As shown in Figure 7-12, to push `'D'` into the stack, first we create a new node and store `'D'` into it. Next, we put the new node on top of the stack. Finally, we make `stackTop` point to the top element of the stack. The definition of the function `push` is as follows:

```
template <class Type>
void linkedStackType<Type>::push(const Type& newElement)
{
    nodeType<Type> *newNode;  //pointer to create the new node

    newNode = new nodeType<Type>; //create the node

    newNode->info = newElement; //store newElement in the node
    newNode->link = stackTop; //insert newNode before stackTop
    stackTop = newNode;       //set stackTop to point to the
                              //top node
} //end push
```

We do not need to check whether the stack is full before we push an element onto the stack because in this implementation, logically, the stack is never full.

Return the Top Element

The operation to return the top element of the stack is quite straightforward. Its definition is as follows:

```
template <class Type>
Type linkedStackType<Type>::top() const
{
    assert(stackTop != NULL); //if stack is empty,
                              //terminate the program
    return stackTop->info;    //return the top element
}//end top
```

Pop

Now we consider the **pop** operation, which removes the top element of the stack. Consider the stack shown in Figure 7-13.

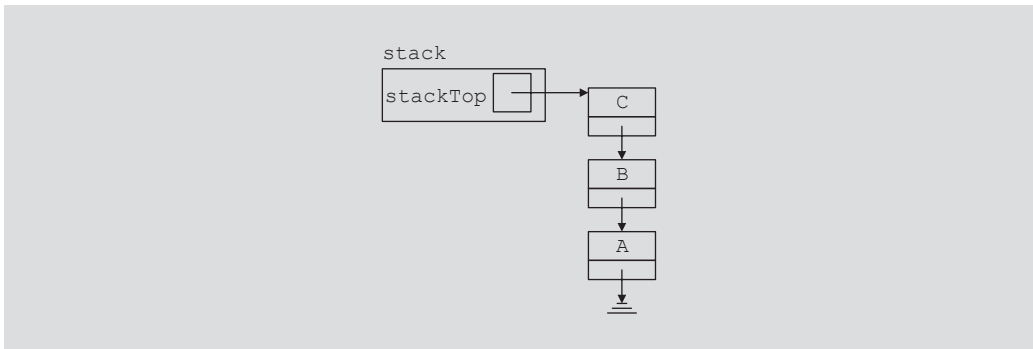


FIGURE 7-13 Stack before the pop operation

Figure 7-14 shows the **pop** operation.

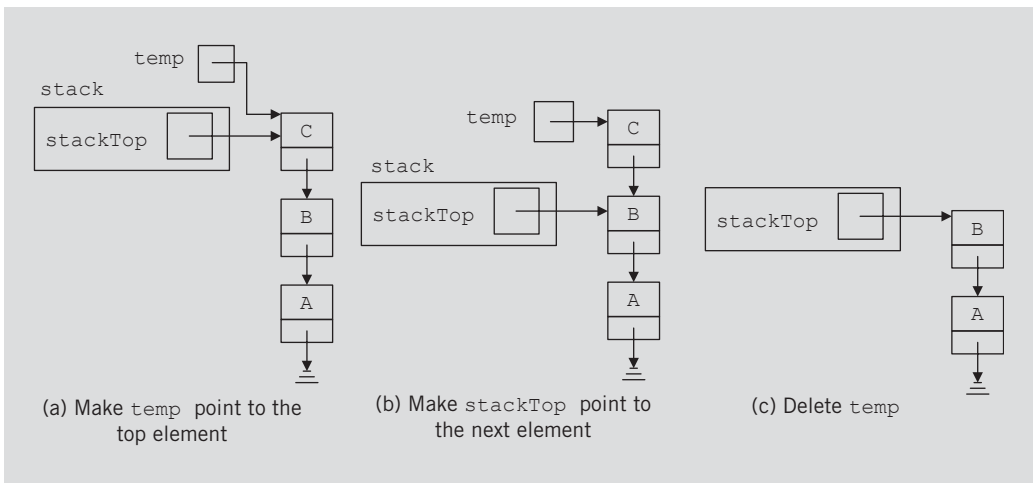


FIGURE 7-14 Pop operation

As shown in Figure 7-14, first we make a pointer `temp` point to the top of the stack. Next we make `stackTop` point to the next element of the stack, which will become the top element of the stack. Finally, we delete `temp`. The definition of the function `pop` is as follows:

```
template <class Type>
void linkedStackType<Type>::pop()
{
    nodeType<Type> *temp;   //pointer to deallocate memory

    if (stackTop != NULL)
    {
        temp = stackTop;  //set temp to point to the top node

        stackTop = stackTop->link;  //advance stackTop to the
                                    //next node
        delete temp;    //delete the top node
    }
    else
        cout << "Cannot remove from an empty stack." << endl;
}//end pop
```

Copy Stack

The function `copyStack` makes an identical copy of a stack. Its definition is similar to the definition of `copyList` for linked lists, given in Chapter 5. The definition of the function `copyStack` is as follows:

```
template  <class Type>
void linkedStackType<Type>::copyStack
                     (const linkedStackType<Type>& otherStack)
{
    nodeType<Type> *newNode, *current, *last;

    if (stackTop != NULL) //if stack is nonempty, make it empty
        initializeStack();

    if (otherStack.stackTop == NULL)
        stackTop = NULL;
    else
    {
        current = otherStack.stackTop;  //set current to point
                                    //to the stack to be copied

            //copy the stackTop element of the stack
        stackTop = new nodeType<Type>;  //create the node

        stackTop->info = current->info; //copy the info
        stackTop->link = NULL;  //set the link field to NULL
        last = stackTop;        //set last to point to the node
        current = current->link; //set current to point to the
                                 //next node
```

```
            //copy the remaining stack
        while (current != NULL)
        {
            newNode = new nodeType<Type>;

            newNode->info = current->info;
            newNode->link = NULL;
            last->link = newNode;
            last = newNode;
            current = current->link;
        }//end while
    }//end else
} //end copyStack
```

Constructors and Destructors

We have already discussed the default constructor. To complete the implementation of the stack operations, next we give the definitions of the functions to implement the copy constructor and the destructor, and to overload the assignment operator. (These functions are similar to those discussed for linked lists in Chapter 5.)


```
    //copy constructor
template <class Type>
linkedStackType<Type>::linkedStackType(
                      const linkedStackType<Type>& otherStack)
{
    stackTop = NULL;
    copyStack(otherStack);
}//end copy constructor

    //destructor
template  <class Type>
linkedStackType<Type>::~linkedStackType()
{
    initializeStack();
}//end destructor
```

Overloading the Assignment Operator (=)

The definition of the function to overload the assignment operator for the `class linkedStackType` is as follows:

```
template <class Type>
const linkedStackType<Type>& linkedStackType<Type>::operator=
                      (const linkedStackType<Type>& otherStack)
{
    if (this != &otherStack) //avoid self-copy
        copyStack(otherStack);

    return *this;
}//end operator=
```

Table 7-2 summarizes the time complexity of the operations to implement a linked stack.

TABLE 7-2 Time complexity of the operations of the `class linkedStackType` on a stack with *n* elements

Function	Time complexity
`isEmptyStack`	$O(1)$
`isFullStack`	$O(1)$
`initializeStack`	$O(n)$
constructor	$O(1)$
`top`	$O(1)$
`push`	$O(1)$
`pop`	$O(1)$
`copyStack`	$O(n)$
destructor	$O(n)$
copy constructor	$O(n)$
Overloading the assignment operator	$O(n)$

The definition of a stack, and the functions to implement the stack operations discussed previously, are generic. Also, as in the case of an array representation of a stack, in the linked representation of a stack, we put the definition of the stack and the functions to implement the stack operations together in a (header) file. A client's program can include this header file via the `include` statement.

The program in Example 7-3 illustrates how a `linkedStack` object is used in a program.

EXAMPLE 7-3

We assume that the definition of the `class linkedStackType` and the functions to implement the stack operations are included in the header file `"linkedStack.h"`.

```
//***********************************************************
// Author: D.S. Malik
//
// This program tests various operations of a linked stack.
//***********************************************************
```

```
#include <iostream>
#include "linkedStack.h"

using namespace std;

void testCopy(linkedStackType<int> OStack);

int main()
{
    linkedStackType<int> stack;
    linkedStackType<int> otherStack;
    linkedStackType<int> newStack;

        //Add elements into stack
    stack.push(34);
    stack.push(43);
    stack.push(27);

        //Use the assignment operator to copy the elements
        //of stack into newStack
    newStack = stack;

    cout << "After the assignment operator, newStack: "
         << endl;

        //Output the elements of newStack
    while (!newStack.isEmptyStack())
    {
        cout << newStack.top() << endl;
        newStack.pop();
    }

        //Use the assignment operator to copy the elements
        //of stack into otherStack
    otherStack = stack;

    cout << "Testing the copy constructor." << endl;

    testCopy(otherStack);

    cout << "After the copy constructor, otherStack: " << endl;

    while (!otherStack.isEmptyStack())
    {
        cout << otherStack.top() << endl;
        otherStack.pop();
    }

    return 0;
}
```

```
     //Function to test the copy constructor
void testCopy(linkedStackType<int> OStack)
{
    cout << "Stack in the function testCopy:" << endl;

    while (!OStack.isEmptyStack())
    {
        cout << OStack.top() << endl;
        OStack.pop();
    }
}
```

Sample Run:

```
After the assignment operator, newStack:
27
43
34
Testing the copy constructor.
Stack in the function testCopy:
27
43
34
After the copy constructor, otherStack:
27
43
34
```

Stack as Derived from the `class unorderedLinkedList`

If we compare the `push` function of the stack with the `insertFirst` function discussed for general lists in Chapter 5, we see that the algorithms to implement these operations are similar. A comparison of other functions, such as `initializeStack` and `initializeList`, `isEmptyList` and `isEmptyStack`, and so on, suggests that the `class linkedStackType` can be derived from the `class linkedListType`. Moreover, the functions `pop` and `isFullStack` can be implemented as in the previous section. Note that the `class linkedListType` is an abstract class and does not implement all the operations. However, the `class unorderedLinkedList` is derived from the the `class linkedListType` and provides the definitions of the abstract functions of the `class linkedListType`. Therefore, we can derive the `class linkedStackType` from the `class unorderedLinkedList`.

Next, we define the `class linkedStackType` that is derived from the `class unorderedLinkedList`. The definitions of the functions to implement the stack operations are also given.

```
#include <iostream>
#include "unorderedLinkedList.h"
```

```
using namespace std;

template <class Type>
class linkedStackType: public unorderedLinkedList<Type>
{
public:
    void initializeStack();
    bool isEmptyStack() const;
    bool isFullStack() const;
    void push(const Type& newItem);
    Type top() const;
    void pop();
};

template <class Type>
void linkedStackType<Type>::initializeStack()
{
    unorderedLinkedList<Type>::initializeList();
}

template <class Type>
bool linkedStackType<Type>::isEmptyStack() const
{
    return unorderedLinkedList<Type>::isEmptyList();
}

template <class Type>
bool linkedStackType<Type>::isFullStack() const
{
    return false;
}

template <class Type>
void linkedStackType<Type>::push(const Type& newElement)
{
    unorderedLinkedList<Type>::insertFirst(newElement);
}

template <class Type>
Type linkedStackType<Type>::top() const
{
    return unorderedLinkedList<Type>::front();
}

template <class Type>
void linkedStackType<Type>::pop()
{
    nodeType<Type> *temp;

    temp = first;
    first = first->link;
    delete temp;
}
```

Application of Stacks: Postfix Expressions Calculator

The usual notation for writing arithmetic expressions (the notation we learned in elementary school) is called **infix** notation, in which the operator is written between the operands. For example, in the expression *a* + *b*, the operator + is between the operands *a* and *b*. In infix notation, the operators have precedence. That is, we must evaluate expressions from left to right, and multiplication and division have higher precedence than addition and subtraction. If we want to evaluate the expression in a different order, we must include parentheses. For example, in the expression *a* + *b* ⋆ *c*, we first evaluate ⋆ using the operands *b* and *c*, and then we evaluate + using the operand *a* and the result of *b* ⋆ c.

In the early 1920s, the Polish mathematician Jan Lukasiewicz discovered that if operators were written before the operands (**prefix** or **Polish** notation; for example, + *a b*), the parentheses can be omitted. In the late 1950s, the Australian philosopher and early computer scientist Charles L. Hamblin proposed a scheme in which the operators *follow* the operands (postfix operators), resulting in the **Reverse Polish** notation. This has the advantage that the operators appear in the order required for computation.

For example, the expression:

a + *b* ⋆ *c*

in a postfix expression is:

a b c ⋆ +

The following example shows various infix expressions and their equivalent postfix expressions.

EXAMPLE 7-4

Infix expression	Equivalent postfix expression
a + *b*	*a b* +
a + *b* * *c*	*a b c* * +
a * *b* + *c*	*a b* * *c* +
(*a* + *b*) * *c*	*a b* + *c* *
(*a* − *b*) * (*c* + *d*)	*a b* − *c d* + *
(*a* + *b*) * (*c* − *d* / *e*) + *f*	*a b* + *c d e* / − * *f* +

Shortly after Lukasiewicz's discovery, it was realized that postfix notation had important applications in computer science. In fact, many compilers use stacks to first translate infix expressions into some form of postfix notation and then translate this postfix expression into machine code. Postfix expressions can be evaluated using the following algorithm:

Scan the expression from left to right. When an operator is found, back up to get the required number of operands, perform the operation, and continue.

Consider the following postfix expression:

```
6 3 + 2 * =
```

Let us evaluate this expression using a stack and the previous algorithm. Figure 7-15 shows how this expression gets evaluated.

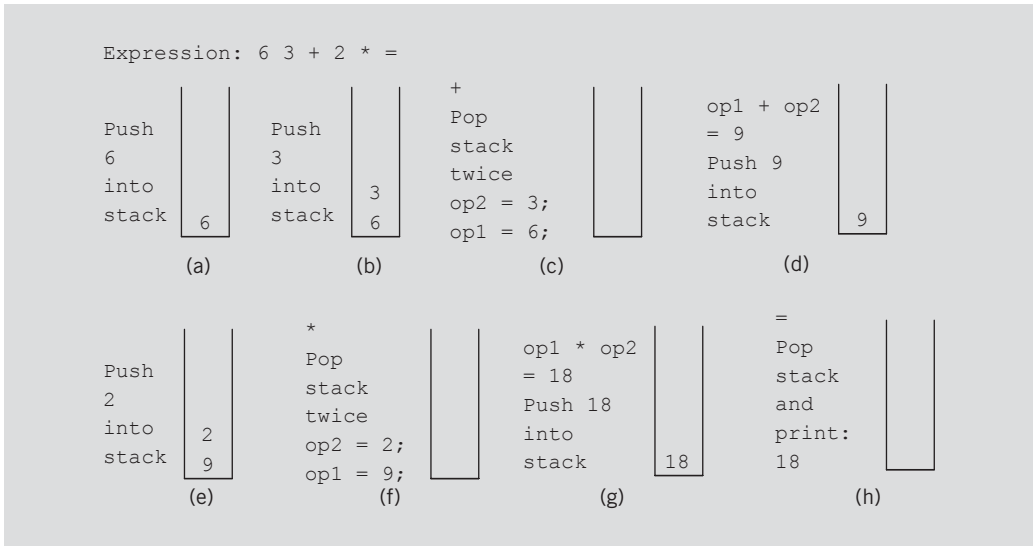


FIGURE 7-15 Evaluating the postfix expression: `6 3 + 2 * =`

Read the first symbol, `6`, which is a number. Push the number onto the stack; see Figure 7-15(a). Read the next symbol, `3`, which is a number. Push the number onto the stack; see Figure 7-15(b). Read the next symbol, `+`, which is an operator. Because an operator requires two operands to be evaluated, pop the stack twice; see Figure 7-15(c). Perform the operation and put the result back onto the stack; see Figure 7-15(d).

Read the next symbol, `2`, which is a number. Push the number onto the stack; see Figure 7-15(e). Read the next symbol, `*`, which is an operator. Because an operator requires two operands to be evaluated, pop the stack twice; see Figure 7-15(f). Perform the operation and put the result back onto the stack; see Figure 7-15(g).

Scan the next symbol, `=`, which is the equal sign, indicating the end of the expression. Therefore, print the result. The result of the expression is in the stack, so pop and print; see Figure 7-15(h).

The value of the expression `6 3 + 2 * = 18`.

From this discussion, it is clear that when we read a symbol other than a number, the following cases arise:

1. The symbol we read is one of the following: +, -, *, /, or =.
   a. If the symbol is +, -, *, or /, the symbol is an operator and so we must evaluate it. Because an operator requires two operands, the stack must have at least two elements; otherwise, the expression has an error.
   b. If the symbol is = (an equal sign), the expression ends and we must print the answer. At this step, the stack must contain exactly one element; otherwise, the expression has an error.
2. The symbol we read is something other than +, -, *, /, or =. In this case, the expression contains an illegal operator.

It is also clear that when an operand (number) is encountered in an expression, it is pushed onto the stack because the operator comes after the operands.

Consider the following expressions:

i. `7 6 + 3 ; 6 - =`
ii. `14 + 2 3 * =`
iii. `14 2 3 + =`

Expression (i) has an illegal operator, expression (ii) does not have enough operands for +, and expression (iii) has too many operands. In the case of expression (iii), when we encounter the equal sign (=), the stack will have two elements and this error cannot be discovered until we are ready to print the value of the expression.

To make the input easier to read, we assume that the postfix expressions are in the following form:

```
#6 #3 + #2 * =
```

The symbol # precedes each number in the expression. If the symbol scanned is #, the next input is a number (that is, an operand). If the symbol scanned is not #, it is either an operator (might be illegal) or an equal sign (indicating the end of the expression). Furthermore, we assume that each expression contains only the +, -, *, and / operators.

This program outputs the entire postfix expression together with the answer. If the expression has an error, the expression is discarded. In this case, the program outputs the expression together with an appropriate error message. Because an expression might contain an error, we must clear the stack before processing the next expression. Also, the stack must be initialized; that is, the stack must be empty.

MAIN ALGORITHM

Pursuant to the previous discussion, the main algorithm in pseudocode is as follows:

```
Read the first character
while not the end of input data
{
    a. initialize the stack
    b. process the expression
    c. output result
    d. get the next expression
}
```

To simplify the complexity of the function `main`, we write four functions—`evaluateExpression`, `evaluateOpr`, `discardExp`, and `printResult`. The function `evaluateExpression`, if possible, evaluates the expression and leaves the result in the stack. If the postfix expression is error free, the function `printResult` outputs the result. The function `evaluateOpr` evaluates an operator, and the function `discardExp` discards the current expression if there is any error in the expression.

FUNCTION `evaluateExpression`

The function `evaluateExpression` evaluates each postfix expression. Each expression ends with the symbol `=`. The general algorithm in pseudocode is as follows:

```
while (ch is not = '=') //process each expression
                        //= marks the end of an expression
{
    switch (ch)
    {
    case '#':
        read a number
        output the number;
        push the number onto the stack;
        break;
     default:
        assume that ch is an operation
        evaluate the operation;
    } //end switch

    if no error was found, then
    {
        read next ch;
        output ch;
    }
    else
        Discard the expression
} //end while
```

From this algorithm, it follows that this method has five parameters—a parameter to access the input file, a parameter to access the output file, a parameter to access the stack, a parameter to pass a character of the expression, and a parameter to indicate whether there is an error in the expression. The definition of this function is as follows:

```
void evaluateExpression(ifstream& inpF, ofstream& outF,
                        stackType<double>& stack,
                        char& ch, bool& isExpOk)
```

```
{
    double num;
    outF << ch;

    while (ch != '=')
    {
        switch (ch)
        {
        case '#':
            inpF >> num;
            outF << num << " ";
            if (!stack.isFullStack())
                stack.push(num);
            else
            {
                cout << "Stack overflow. "
                     << "Program terminates!" << endl;
                exit(0);  //terminate the program
            }

            break;

        default:
            evaluateOpr(outF, stack, ch, isExpOk);
        }//end switch

        if (isExpOk) //if no error
        {
            inpF >> ch;
            outF << ch;

            if (ch != '#')
                outF << " ";
        }
        else
            discardExp(inpF, outF, ch);
    } //end while (!= '=')
} //end evaluateExpression
```

Note that the funtion **exit** terminates the program.

FUNCTION evaluateOpr

This function (if possible) evaluates an expression. Two operands are needed to evaluate an operation and operands are saved in the stack. Therefore, the stack must contain at least two numbers. If the stack contains fewer than two numbers, the expression has an error. In this case, the entire expression is discarded and an appropriate message is printed. This function also checks for any illegal operations. In pseudocode, this function is as follows:

```
if stack is empty
{
    error in the expression
    set expressionOk to false
}
else
{
    retrieve the top element of stack into op2
    pop stack
    if stack is empty
    {
        error in the expression
        set expressionOk to false
    }
    else
    {
        retrieve the top element of stack into op1
        pop stack

          //If the operation is legal, perform the operation and
          //push the result onto the stack;
          //otherwise, report the error.
        switch (ch)
        {
        case '+': //add the operands: op1 + op2
            stack.push(op1 + op2);
            break;
        case '-': //subtract the operands: op1 - op2
            stack.push(op1 - op2);
            break;
        case '*': //multiply the operands: op1 * op2
            stack.push(op1 * op2);
            break;
        case '/': //If (op2 != 0), op1 / op2
            stack.push(op1 / op2);
            break;
        otherwise operation is illegal
           {
               output an appropriate message;
               set expressionOk to false
           }
        } //end switch
}
```

Following this pseudocode, the definition of the function `evaluateOpr` is as follows:

```
void evaluateOpr(ofstream& out, stackType<double>& stack,
                 char& ch, bool& isExpOk)
{
    double op1, op2;
```

```
    if (stack.isEmptyStack())
    {
        out << " (Not enough operands)";
        isExpOk = false;
    }
    else
    {
        op2 = stack.top();
        stack.pop();

        if (stack.isEmptyStack())
        {
            out << " (Not enough operands)";
            isExpOk = false;
        }
        else
        {
            op1 = stack.top();
            stack.pop();

            switch (ch)
            {
            case '+':
                stack.push(op1 + op2);
                break;

            case '-':
                stack.push(op1 - op2);
                break;

            case '*':
                stack.push(op1 * op2);
                break;

            case '/':
                if (op2 != 0)
                    stack.push(op1 / op2);
                else
                {
                    out << " (Division by 0)";
                    isExpOk = false;
                }
                break;

            default:
                out << " (Illegal operator)";
                isExpOk = false;
            }//end switch
        } //end else
    } //end else
} //end evaluateOpr
```

FUNCTION discardExp

This function is called whenever an error is discovered in the expression. It reads and writes the input data only until the input is '=', the end of the expression. The definiton of this function is as follows:

```
void discardExp(ifstream& in, ofstream& out, char& ch)
{
    while (ch != '=')
    {
        in.get(ch);
        out << ch;
    }
} //end discardExp
```

FUNCTION printResult

If the postfix expression contains no errors, the function printResult prints the result; otherwise, it outputs an appropriate message. The result of the expression is in the stack and the output is sent to a file. Therefore, this function must have access to the stack and the output file. Suppose that no errors were encountered by the method evaluateExpression. If the stack has only one element, the expression is error free and the top element of the stack is printed. If either the stack is empty or it has more than one element, there is an error in the postfix expression. In this case, this method outputs an appropriate error message. The definition of this method is as follows:


```
void printResult(ofstream& outF, stackType<double>& stack,
                 bool isExpOk)
{
    double result;

    if (isExpOk) //if no error, print the result
    {
        if (!stack.isEmptyStack())
        {
            result = stack.top();
            stack.pop();

            if (stack.isEmptyStack())
                outF << result << endl;
            else
                outF << " (Error: Too many operands)" << endl;
        } //end if
        else
            outF << " (Error in the expression)" << endl;
    }
    else
        outF << " (Error in the expression)" << endl;

    outF << "_____________________________________"
         << endl << endl;
} //end printResult
```

PROGRAM LISTING

```
//***********************************************************
// Author: D.S. Malik
//
// This program uses a stack to evaluate postfix expressions.
//***********************************************************

#include <iostream>
#include <iomanip>
#include <fstream>
#include "mystack.h"

using namespace std;

void evaluateExpression(ifstream& inpF, ofstream& outF,
                        stackType<double>& stack,
                        char& ch, bool& isExpOk);
void evaluateOpr(ofstream& out, stackType<double>& stack,
                 char& ch, bool& isExpOk);
void discardExp(ifstream& in, ofstream& out, char& ch);
void printResult(ofstream& outF, stackType<double>& stack,
                 bool isExpOk);

int main()
{
    bool expressionOk;
    char ch;
    stackType<double> stack(100);
    ifstream infile;
    ofstream outfile;

    infile.open("RpnData.txt");

    if (!infile)
    {
        cout << "Cannot open the input file. "
             << "Program terminates!" << endl;
        return 1;
    }

    outfile.open("RpnOutput.txt");

    outfile << fixed << showpoint;
    outfile << setprecision(2);

    infile >> ch;
    while (infile)
    {
        stack.initializeStack();
        expressionOk = true;
        outfile << ch;
```

```
        evaluateExpression(infile, outfile, stack, ch,
                           expressionOk);
        printResult(outfile, stack, expressionOk);
        infile >> ch; //begin processing the next expression
    } //end while

    infile.close();
    outfile.close();

    return 0;

} //end main

//Place the definitions of the function evaluateExpression,
//evaluateOpr, discardExp, and printResult as described
//previously here.
```

Sample Run:

Input File

```
#35 #27 + #3 * =
#26 #28 + #32 #2 ; - #5 / =
#23 #30 #15 * / =
#2 #3 #4 + =
#20 #29 #9 * ; =
#25 #23 - + =
#34 #24 #12 #7 / * + #23 - =
```

Output

```
#35.00 #27.00 + #3.00 * = 186.00
_____________________________________

#26.00 #28.00 + #32.00 #2.00 ; (Illegal operator) - #5 / = (Error in the expression)
_____________________________________

#23.00 #30.00 #15.00 * / = 0.05
_____________________________________

#2.00 #3.00 #4.00 + = (Error: Too many operands)
_____________________________________

#20.00 #29.00 #9.00 * ; (Illegal operator) = (Error in the expression)
_____________________________________

#25.00 #23.00 - + (Not enough operands) = (Error in the expression)
_____________________________________

#34.00 #24.00 #12.00 #7.00 / * + #23.00 - = 52.14
_____________________________________
```

Removing Recursion: Nonrecursive Algorithm to Print a Linked List Backward

In Chapter 6, we used recursion to print a linked list backward. In this section, you will learn how a stack can be used to design a nonrecursive algorithm to print a linked list backward.

Consider the linked list shown in Figure 7-16.

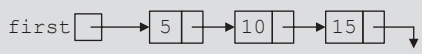


FIGURE 7-16 Linked list

To print the list backward, first we need to get to the last node of the list, which we can do by traversing the linked list starting at the first node. However, once we are at the last node, how do we get back to the previous node, especially given that links go in only one direction? You can again traverse the linked list with the appropriate loop termination condition, but this approach might waste a considerable amount of computer time, especially if the list is very large. Moreover, if we do this for every node in the list, the program might execute very slowly. Next, we show how to use a stack effectively to print the list backward.

After printing the `info` of a particular node, we need to move to the node immediately behind this node. For example, after printing `15`, we need to move to the node with `info` `10`. Thus, while initially traversing the list to move to the last node, we must save a pointer to each node. For example, for the list in Figure 7-16, we must save a pointer to each of the nodes with `info` `5` and `10`. After printing `15`, we go back to the node with `info` `10`; after printing `10`, we go back to the node with `info` `5`. From this, it follows that we must save pointers to each node in a stack, so as to implement the Last In First Out principle.

Because the number of nodes in a linked list is usually not known, we will use the linked implementation of a stack. Suppose that `stack` is an object of type `linkedListType`, and `current` is a pointer of the same type as the pointer `first`. Consider the following statements:

```
current = first;                //Line 1

while (current != NULL)         //Line 2
{                               //Line 3
    stack.push(current);        //Line 4
    current = current->link;    //Line 5
}                               //Line 6
```

After the statement in Line 1 executes, **current** points to the first node. (See Figure 7-17)

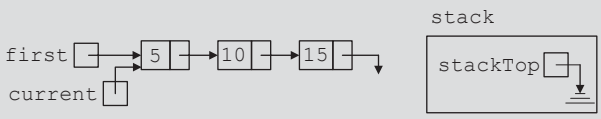


FIGURE 7-17 List after the statement `current = first;` executes

Because **current** is not **NULL**, the statements in Lines 4 and 5 execute. (See Figure 7-18.)

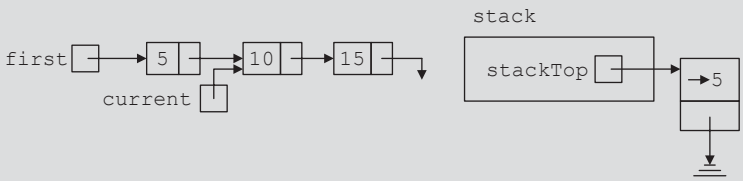


FIGURE 7-18 List and stack after the statements `stack.push(current);` and `current = current->link;` execute

Because **current** is not **NULL**, statements in Lines 4 and 5 execute. In fact, statements in Lines 4 and 5 execute until **current** beomes **NULL**. When **current** is **NULL**, Figure 7-19 results.

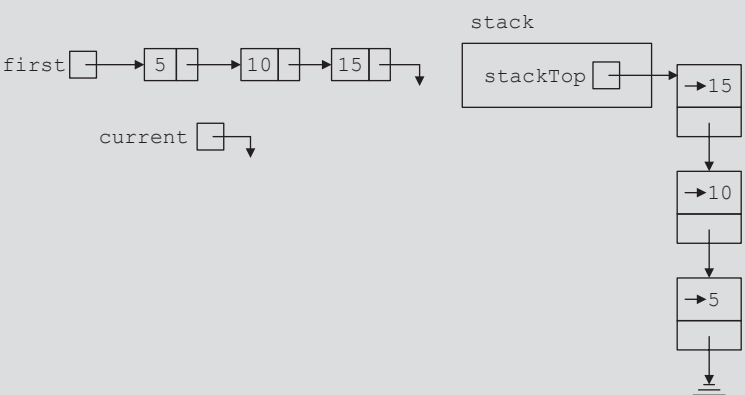


FIGURE 7-19 List and stack after the `while` statement executes

7

After the statement in Line 4 executes, the loop condition, in Line 2, is evaluated again. Because `current` is `NULL`, the loop condition evaluates to `false` and the `while` loop, in Line 2, terminates. From Figure 7-19, it follows that a pointer to each node in the linked list is saved in the stack. The top element of the stack contains a pointer to the last node in the list, and so on. Let us now execute the following statements:

```
while (!stack.isEmptyStack())        //Line 7
{                                    //Line 8
    current = stack.top();           //Line 9
    stack.pop();                     //Line 10
    cout << current->info << " ";    //Line 11
}                                    //Line 12
```

The loop condition in Line 7 evaluates to `true` because the stack is nonempty. Therefore, the statements in Lines 9, 10, and 11 execute. After the statement in Line 9 executes, `current` points to the last node. The statement in Line 10 removes the top element of the stack; see Figure 7-20.

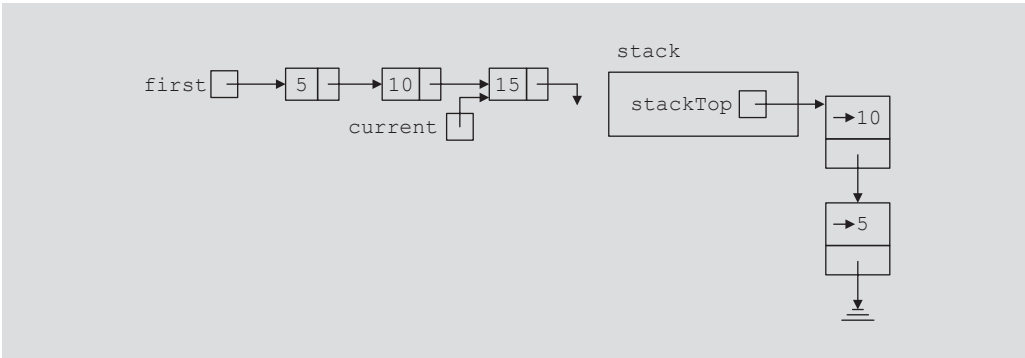


FIGURE 7-20 List and stack after the statements `current = stack.top();` and `stack.pop();` execute

The statement in Line 11 outputs `current->info`, which is `15`.

Because `stack` is nonempty, the body of the `while` loop executes again. In fact, for the linked list in Figure 7-20, the body of the `while` loop executes two more times; the first time it prints `10`, and the second time it prints `5`. Ater printing `5`, the `stack` becomes empty and the `while` loop terminates. It follows that the `while` loop in Line 7 produces the following output:

```
15 10 5
```

STL `class stack`

The previous sections discussed the data structure `stack` in detail. Because a stack is an important data structure, the Standard Template Library (STL) provides a class to implement a stack in a program. The name of the class defining a stack is `stack`; the name of

the header file containing the definition of the **class stack** is **stack**. The implementation of the **class stack** provided by the STL is similar to the one described in this chapter. Table 7-3 defines the various operations supported by the stack container class.

TABLE 7-3 Operations on a `stack` object

Operation	Effect
`size`	Returns the actual number of elements in the stack.
`empty`	Returns **true** if the stack is empty, and **false** otherwise.
`push(item)`	Inserts a copy of item into the stack.
`top`	Returns the top element of the stack, but does not remove the top element from the stack. This operation is implemented as a value-returning function.
`pop`	Removes the top element of the stack.

In addition to the operations **size**, **empty**, **push**, **top**, and **pop**, the stack container class provides relational operators to compare two stacks. For example, the relational operator **==** can be used to determine whether two stacks are identical.

The program in Example 7-5 illustrates how to use the stack container class.

EXAMPLE 7-5

```
//***********************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the STL class stack in
// a program.
//***********************************************************

#include <iostream>                                    //Line 1
#include <stack>                                       //Line 2

using namespace std;                                   //Line 3

int main()                                             //Line 4
{                                                      //Line 5
    stack<int>  intStack;                              //Line 6
```

```
    intStack.push(16);                                  //Line 7
    intStack.push(8);                                   //Line 8
    intStack.push(20);                                  //Line 9
    intStack.push(3);                                   //Line 10

    cout << "Line 11: The top element of intStack: "
         << intStack.top() << endl;                     //Line 11

    intStack.pop();                                     //Line 12

    cout << "Line 13: After the pop operation, the "
         << " top element of intStack: "
         << intStack.top() << endl;                     //Line 13

    cout << "Line 14: intStack elements: ";             //Line 14

    while (!intStack.empty())                           //Line 15
    {                                                   //Line 16
        cout << intStack.top() << " ";                  //Line 17
        intStack.pop();                                 //Line 18
    }                                                   //Line 19

    cout << endl;                                       //Line 20

    return 0;                                           //Line 21
}                                                       //Line 22
```

Sample Run:

```
Line 11: The top element of intStack: 3
Line 13: After the pop operation, the top element of intStack: 20
Line 14: intStack elements: 20 8 16
```

The preceding output is self-explanatory. The details are left as an exercise for you.

QUICK REVIEW

1. A stack is a data structure in which the items are added and deleted from one end only.
2. A stack is a Last In First Out (LIFO) data structure.
3. The basic operations on a stack are as follows: Push an item onto the stack, pop an item from the stack, retrieve the top element of the stack, initialize the stack, check whether the stack is empty, and check whether the stack is full.
4. A stack can be implemented as an array or a linked list.
5. The middle elements of a stack should not be accessed directly.
6. Stacks are restricted versions of arrays and linked lists.

7. Postfix notation does not require the use of parentheses to enforce operator precedence.
8. In postfix notation, the operators are written after the operands.
9. Postfix expressions are evaluated according to the following rules:
   a. Scan the expression from left to right.
   b. If an operator is found, back up to get the required number of operands, evaluate the operator, and continue.
10. The STL `class stack` can be used to implement a stack in a program.

EXERCISES

1. Consider the following statements:

```
stackType<int> stack;
int x, y;
```

Show what is output by the following segment of code:

```
x = 4;
y = 0;
stack.push(7);
stack.push(x);
stack.push(x + 5);
y = stack.top();
stack.pop();
stack.push(x + y);
stack.push(y - 2);
stack.push(3);
x = stack.top();
stack.pop();

cout << "x = " << x << endl;
cout << "y = " << y << endl;

while (!stack.isEmptyStack())
{
    cout << stack.top() << endl;
    stack.pop();
}
```

2. Consider the following statements:

```
stackType<int> stack;
int x;
```

Suppose that the input is:

```
14 45 34 23 10 5 -999
```

Show what is output by the following segment of code:

```
stack.push(5);

cin >> x;

while (x != -999)
{
    if (x % 2 == 0)
    {
        if (!stack.isFullStack())
            stack.push(x);
    }
    else
        cout << "x = " << x << endl;
    cin >> x;
}

cout << "Stack Elements: ";

while (!stack.isEmptyStack())
{
    cout << " " << stack.top();
    stack.pop();
}
cout << endl;
```

3. Evaluate the following postfix expressions:
   a. 8 2 + 3 * 16 4 / - =
   b. 12 25 5 1 / / * 8 7 + - =
   c. 70 14 4 5 15 3 / * - - / 6 + =
   d. 3 5 6 * + 13 - 18 2 / + =
4. Convert the following infix expressions to postfix notations:
   a. (A + B) * (C + D) - E
   b. A - (B + C) * D + E / F
   c. ((A + B) / (C - D) + E) * F - G
   d. A + B * (C + D) - E / F * G + H
5. Write the equivalent infix expression for the following postfix expressions:
   a. A B * C +
   b. A B + C D - *
   c. A B - C - D *
6. What is the output of the following program?

```
#include <iostream>
#include <string>
#include "myStack.h"

using namespace std;
```

```
template <class Type>
void mystery(stackType<Type>& s, stackType<Type>& t);

int main()
{
    stackType<string> s1;
    stackType<string> s2;

    string list[] = {"Winter", "Spring", "Summer", "Fall",
                     "Cold", "Warm", "Hot"};

    for (int i = 0; i < 7; i++)
        s1.push(list[i]);

    mystery(s1, s2);

    while (!s2.isEmptyStack())
    {
        cout << s2.top() << " ";
        s2.pop();
    }
    cout << endl;
}

template <class Type>
void mystery(stackType<Type>& s, stackType<Type>& t)
{
    while (!s.isEmptyStack())
    {
        t.push(s.top());
        s.pop();
    }
}
```

7. What is the effect of the following statements? If a statement is invalid, explain why it is invalid. The classes `stackADT`, `stackType`, and `linkedStackType` are as defined in this chapter.

   a. `stackADT<int> newStack;`

   b. `stackType<double> sales(-10);`

   c. `stackType<string> names;`

   d. `linkedStackType<int> numStack(50);`

8. What is the output of the following program?

```
#include <iostream>
#include <string>
#include "myStack.h"

using namespace std;

void mystery(stackType<int>& s, stackType<int>& t);
```

```
int main()
{
    int list[] = {5, 10, 15, 20, 25};

    stackType<int> s1;
    stackType<int> s2;

    for (int i = 0; i < 5; i++)
        s1.push(list[i]);

    mystery(s1, s2);

    while (!s2.isEmptyStack())
    {
        cout << s2.top() << " ";
        s2.pop();
    }
    cout << endl;
}

void mystery(stackType<int>& s, stackType<int>& t)
{
    while (!s.isEmptyStack())
    {
        t.push(2 * s.top());
        s.pop();
    }
}
```

9. What is the output of the following program segment?

```
linkedStackType<int> myStack;

myStack.push(10);
myStack.push(20);
myStack.pop();
cout << myStack.top() << endl;
myStack.push(25);
myStack.push(2 * myStack.top());
myStack.push(-10);
myStack.pop();

linkedStackType<int> tempStack;

tempStack = myStack;

while (!tempStack.isEmptyStack())
{
    cout << tempStack.top() << " ";
    tempStack.pop();
}

cout << endl;

cout << myStack.top() << endl;
```

10. Write the definition of the function template `printListReverse` that uses a stack to print a linked list in reverse order. Assume that this function is a member of the `class linkedListType`, designed in Chapter 5.
11. Write the definition of the function template `second` that takes as a parameter a stack object and returns the second element of the stack. The original stack remains unchanged.
12. Draw the UML class diagram of the `class linkedStackType`.
13. Write the definition of the function template `clear` that takes as a parameter a stack object of the type `stack` (STL class) and removes all the elements from the stack.

PROGRAMMING EXERCISES

1. Two stacks of the same type are the same if they have the same number of elements and their elements at the corresponding positions are the same. Overload the relational operator `==` for the `class stackType` that returns `true` if two stacks of the same type are the same, `false` otherwise. Also, write the definition of the function template to overload this operator.
2. Repeat Exercise 1 for the `class linkedStackType`.
3. a. Add the following operation to the `class stackType`:

   ```
   void reverseStack(stackType<Type> &otherStack);
   ```

   This operation copies the elements of a stack in reverse order onto another stack.

   Consider the following statements:

   ```
   stackType<int> stack1;
   stackType<int> stack2;
   ```

   The statement

   ```
   stack1.reverseStack(stack2);
   ```

   copies the elements of `stack1` onto `stack2` in reverse order. That is, the top element of `stack1` is the bottom element of `stack2`, and so on. The old contents of `stack2` are destroyed and `stack1` is unchanged.

   b. Write the definition of the function `template` to implement the operation `reverseStack`.
4. Repeat Exercises 3a and 3b for the `class linkedStackType`.
5. Write a program that takes as input an arithmetic expression. The program outputs whether the expression contains matching grouping symbols. For example, the arithmetic expressions `{25 + (3 - 6) * 8}` and `7 + 8 * 2` contains matching grouping symbols. However, the expression `5 + {(13 + 7) / 8 - 2 * 9` does not contain matching grouping symbols.

6. Write a program that uses a stack to print the prime factors of a positive integer in descending order.
7. **(Converting a Number from Binary to Decimal)** The language of a computer, called machine language, is a sequence of 0s and 1s. When you press the key A on the keyboard, 01000001 is stored in the computer. Also, the collating sequence of A in the ASCII character set is 65. In fact, the binary representation of A is 01000001 and the decimal representation of A is 65.

   The numbering system we use is called the decimal system, or base 10 system. The numbering system that the computer uses is called the **binary system**, or **base 2 system**. The purpose of this exercise is to write a function to convert a number from base 2 to base 10.

   To convert a number from base 2 to base 10, we first find the weight of each bit in the binary number. The weight of each bit in the binary number is assigned from right to left. The weight of the rightmost bit is 0. The weight of the bit immediately to the left of the rightmost bit is 1, the weight of the bit immediately to the left of it is 2, and so on. Consider the binary number 1001101. The weight of each bit is as follows:

   ```
   weight   6 5 4 3 2 1 0
            1 0 0 1 1 0 1
   ```

   We use the weight of each bit to find the equivalent decimal number. For each bit, we multiply the bit by 2 to the power of its weight, and then we add all of the numbers. For the binary number 1001101, the equivalent decimal number is

   $$1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$
   $$= 64 + 0 + 0 + 8 + 4 + 0 + 1 = 77$$

   To write a program that converts a binary number into the equivalent decimal number, we note two things: (1) The weight of each bit in the binary number must be known, and (2) the weight is assigned from right to left. Because we do not know in advance how many bits are in the binary number, we must process the bits from right to left. After processing a bit, we can add 1 to its weight, giving the weight of the bit immediately to its left. Also, each bit must be extracted from the binary number and multiplied by 2 to the power of its weight. To extract a bit, you can use the mod operator. Write a program that uses a stack to convert a binary number into an equivalent decimal number and test your function for the following values: 11000101, 10101010, 11111111, 10000000, 1111100000.
8. Chapter 6 described how to use recursion to convert a decimal number into an equivalent binary number. Write a program that uses a stack to convert a decimal number into an equivalent binary number.

9. **(Infix to Postfix)** Write a program that converts an infix expression into an equivalent postfix expression.

   The rules to convert an infix expression into an equivalent postfix expression are as follows:

   Suppose `infx` represents the infix expression and `pfx` represents the postfix expression. The rules to convert `infx` into `pfx` are as follows:

   a. Initialize `pfx` to an empty expression and also initialize the stack.
   b. Get the next symbol, `sym`, from `infx`.
      b.1. If `sym` is an operand, append `sym` to `pfx`.
      b.2. If `sym` is `(`, push `sym` into the stack.
      b.3. If `sym` is `)`, pop and append all the symbols from the stack until the most recent left parenthesis. Pop and discard the left parenthesis.
      b.4. If `sym` is an operator:
         b.4.1. Pop and append all the operators from the stack to `pfx` that are above the most recent left parenthesis and have precedence greater than or equal to `sym`.
         b.4.2. Push `sym` onto the stack.
   c. After processing `infx`, some operators might be left in the stack. Pop and append to `pfx` everything from the stack.

   In this program, you will consider the following (binary) arithmetic operators: `+`, `-`, `*`, and `/`. You may assume that the expressions you will process are error free.

   Design a class that stores the infix and postfix strings. The class must include the following operations:

   - **`getInfix`**—Stores the infix expression
   - **`showInfix`**—Outputs the infix expression
   - **`showPostfix`**—Outputs the postfix expression

   Some other operations that you might need are the following:

   - **`convertToPostfix`**—Converts the infix expression into a postfix expression. The resulting postfix expression is stored in `pfx`.
   - **`precedence`**—Determines the precedence between two operators. If the first operator is of higher or equal precedence than the second operator, it returns the value `true`; otherwise, it returns the value `false`.

   Include the constructors and destructors for automatic initialization and dynamic memory deallocation.

Test your program on the following five expressions:

```
A + B - C;
(A + B ) * C;
(A + B) * (C - D);
A + ((B + C) * (E - F) - G) / (H - I);
A + B * (C + D ) - E / F * G + H;
```

For each expression, your answer must be in the following form:

```
Infix Expression: A + B - C;
Postfix Expression: A B + C -
```

10. Redo the program in the section "Application of Stacks: Postfix Expressions Calculator" of this chapter so that it uses the STL `class stack` to evaluate the postfix expressions.
11. Redo Programming Exercise 9 so that it uses the STL `class stack` to convert the infix expressions to postfix expressions.